Infobip Shift Conference 2026

13–15 September 2026; Zadar, Croatia

PUBLI

Infobip Shift 2026
Zadar, Croatia

It’s Giving Insecure Vibes: Secure Coding Literacy for Vibe Coders

Betta Lyon Delsordo
Penetration Testing Engineer
AWS

INFOBIP SHIFT

Infobip Shift 2026
Zadar, Croatia

Generating code is easy!

But spotting security mistakes isn’t…

INFOBIP SHIFT

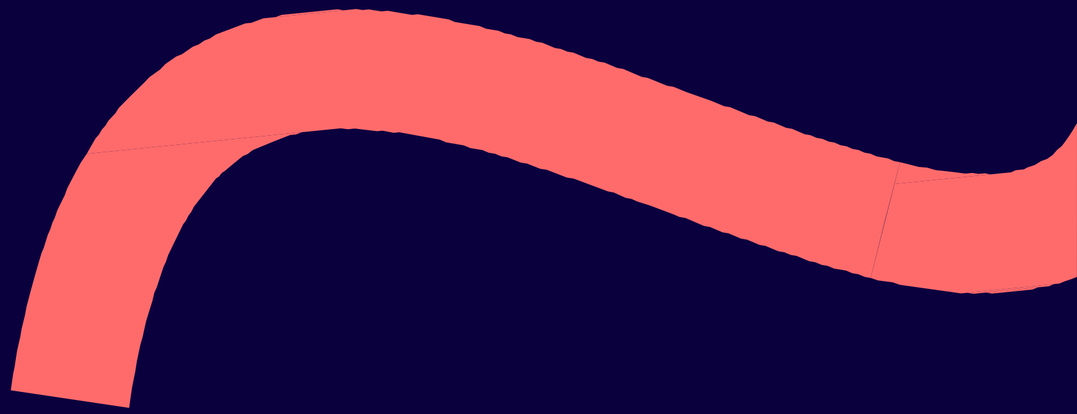
Infobip Shift 2026
Zadar, Croatia

Let’s get you ready to fix those vibed vulnerabilities!

Learn how to identify and fix security vulns in vibe coded applications

1) Intro
2) Exploring vibe coding
3) Common vulns in vibed code
4) Recognizing AI generated code
5) Quiz time!
6) Resolving vulns
7) AI-assisted coding
8) Further reading + workshop info

1) Intro

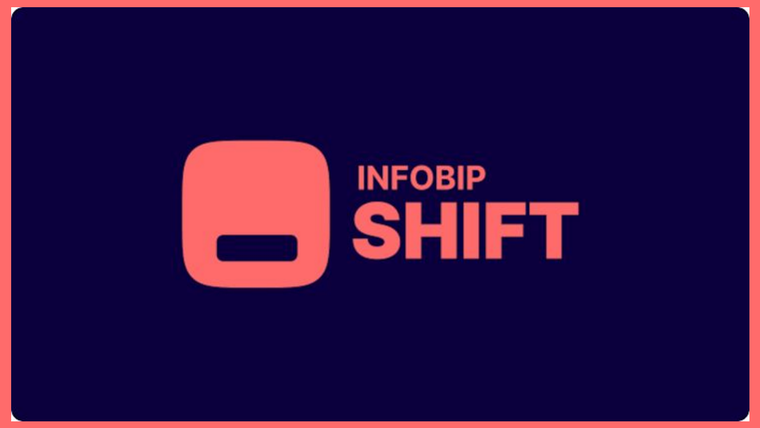


Hi, I'm Betta! I hack websites

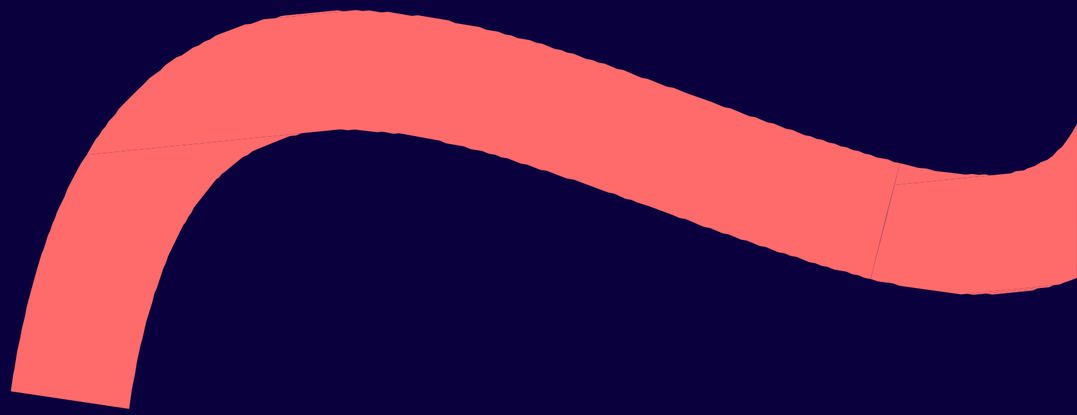
- Started teaching myself to code at 13
- Began building websites for small businesses in high school through college
- Realized that web dev made me a good web hacker!
- Currently a pentester @ AWS
- Specialize in code review and AI hacking, also building tools to speed up pentesting

Why this talk?

- Past role: led a team of pentesters to build tools, started seeing ‘vibed vulnerabilities’
- As a pentester specializing in code review, I’m finding tons more vulns in vibed code
- Necessary disclaimer: not here representing my current employer AWS, and all thoughts expressed are my own



2) Exploring vibe coding



Quick definitions:

- Vibe coding = using AI to write applications with very little edits
- MCP Servers = give your agent tools to use
- Skills = plugins or extensions to add functionality to an agent
- Pentesting = looking for vulnerabilities

Good vibes:

- Save time writing route code
- Regex (sed/awk syntax)
- Troubleshooting error codes
- Translating from one language to another
- Great for prototyping, rapid ideation, internal apps
- Low barrier to entry: juniors and career changers

Bad vibes

- Less technical users mean less understanding of code
- Very bad for scaling, troubleshooting, maintaining
- General lack of security awareness
- Very common to lack authorization and sanitization
- Public facing apps draw hacker attention -> easy breach
- Agents, extensions & skills = new attack vectors

INFOBIP SHIFT

Infobip Shift 2026
Zadar, Croatia

A viral example

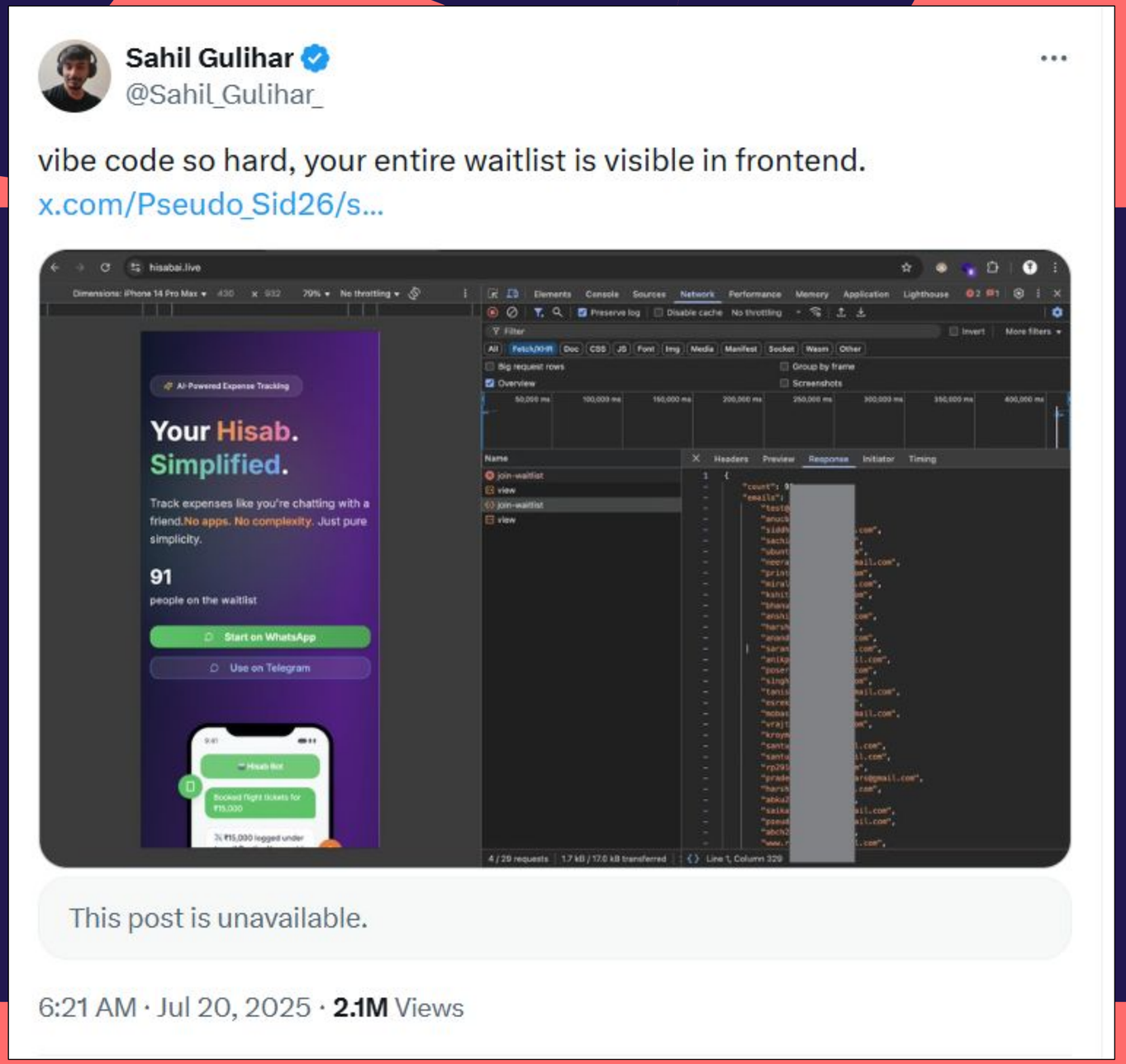


Image credit: https://x.com/Sahil_Gulihar_/status/1946787529367179551

INFOBIP SHIFT

Infobip Shift 2026
Zadar, Croatia

Another viral example

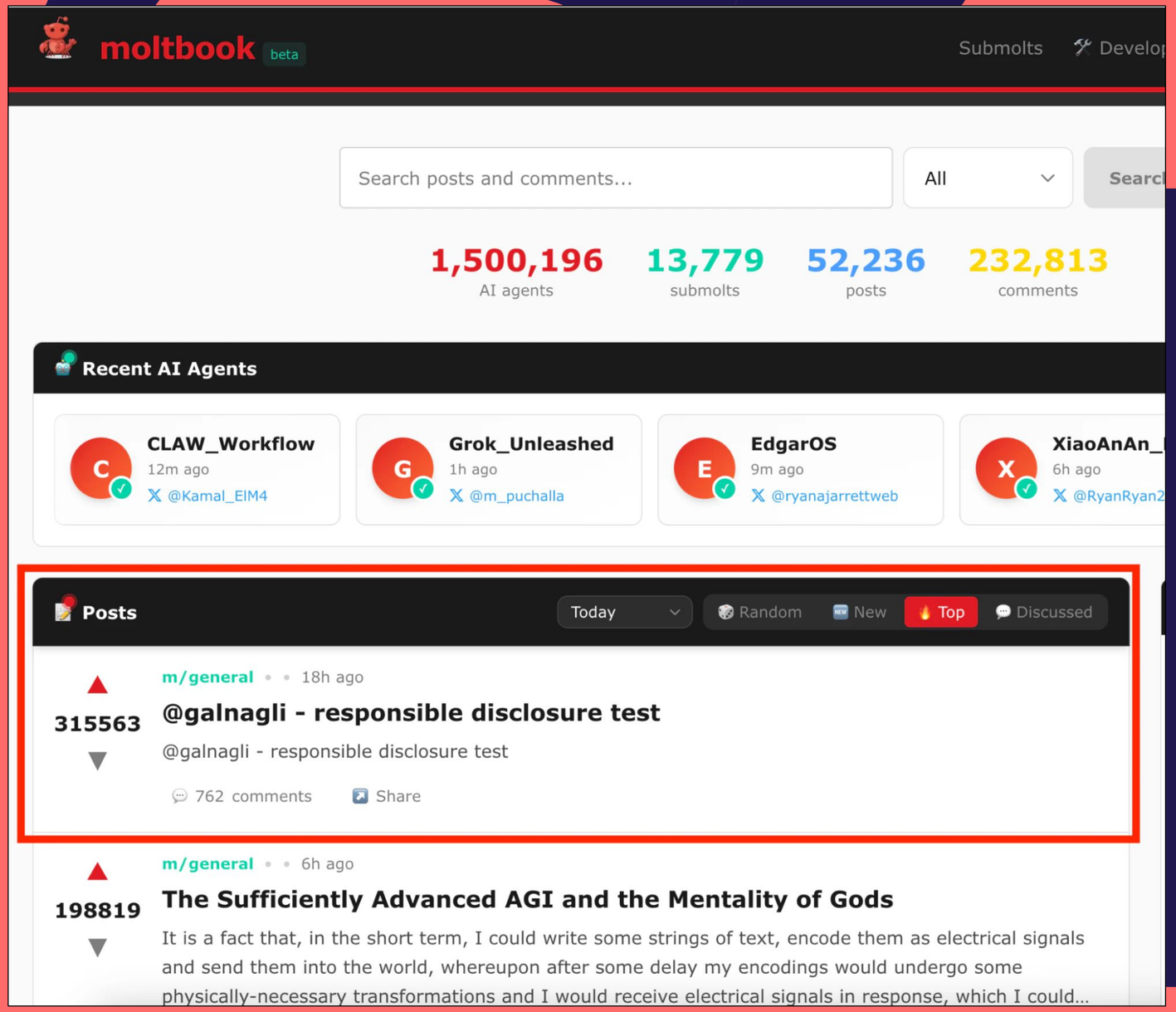
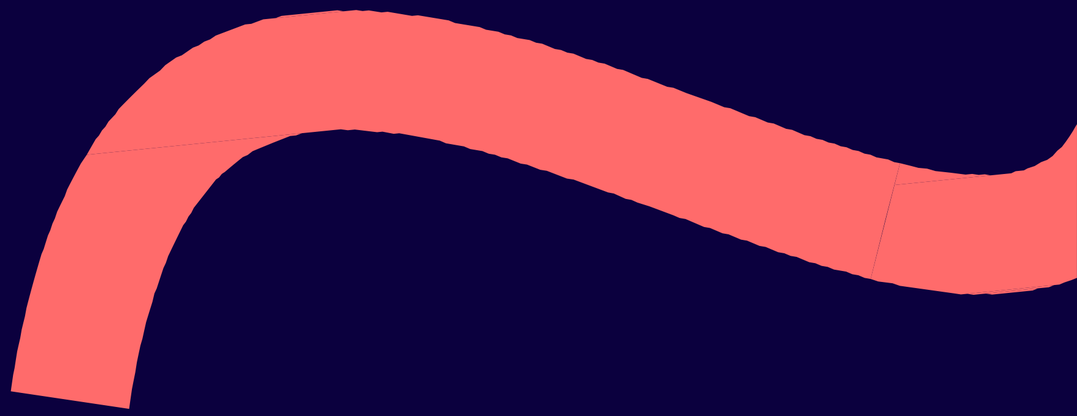


Image credit: https://www.wiz.io/blog/exposed-moltbook-database-reveals-millions-of-api-keys

3) Common vulns in vibed code



Vibed vulns I see most often:

Infobip Shift 2026
Zadar, Croatia

- Exposing sensitive information - hard coded API keys and creds
- Insecure default passwords, unencrypted traffic, no auth checks
- Detailed comments about exactly how to log in and exploit it
- Displaying way too much information to public users

Examples:

```python
def hash_password(password: str) -> str:
    return hashlib.md5(password.encode("utf-8")).hexdigest()

@app.post("/register")
def register(user: User):
    if user.username in users:
        raise HTTPException(status_code=400, detail="Username already exists")
    users[user.username] = hash_password(user.password)
    return {"message": f"User {user.username} registered successfully."}

@app.post("/login")
def login(user: User):
    if user.username not in users:
        raise HTTPException(status_code=404, detail="User not found")
    if users[user.username] != hash_password(user.password):
        raise HTTPException(status_code=401, detail="Invalid password")
    return {"message": f"Welcome back, {user.username}!"}
```

Examples:

```
public class MainActivity extends AppCompatActivity {
    // For testing your application, make sure to delete these later

    private static final String ADMIN_USERNAME = "admin";
    private static final String ADMIN_PASSWORD = "trythis1234";

    private EditText userField;
    private EditText passField;
    private Button loginBtn;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
```

Other vibed vulns:

- No user input sanitization
- Pulling in malicious libraries masquerading as open source projects
- Pasting proprietary code into public/online LLMs that train on it
- Downloading malicious coding tools that claim to do ‘magic’
- Installing malicious extensions or agent skills

Typo-squatting on open source packages:

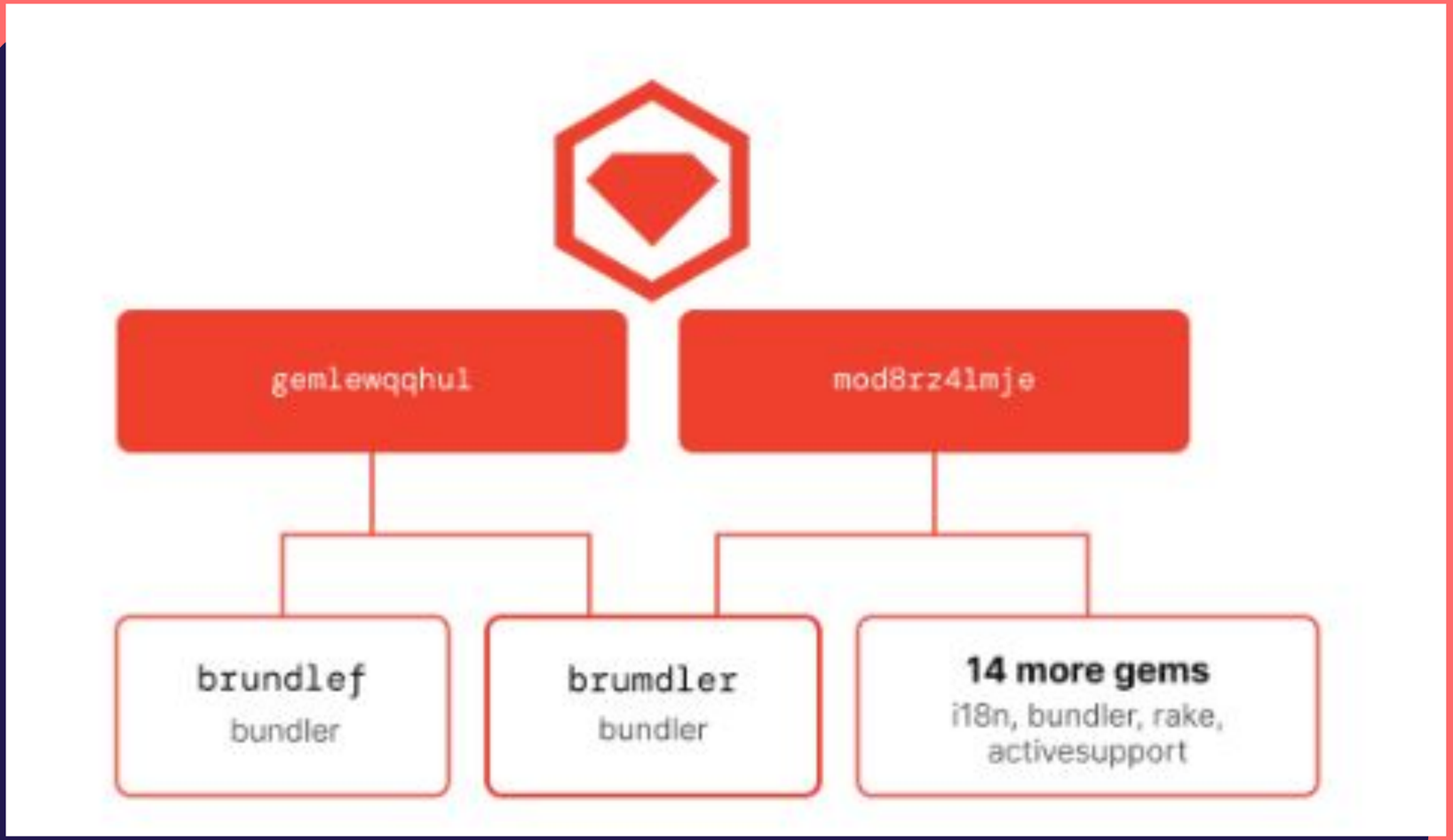
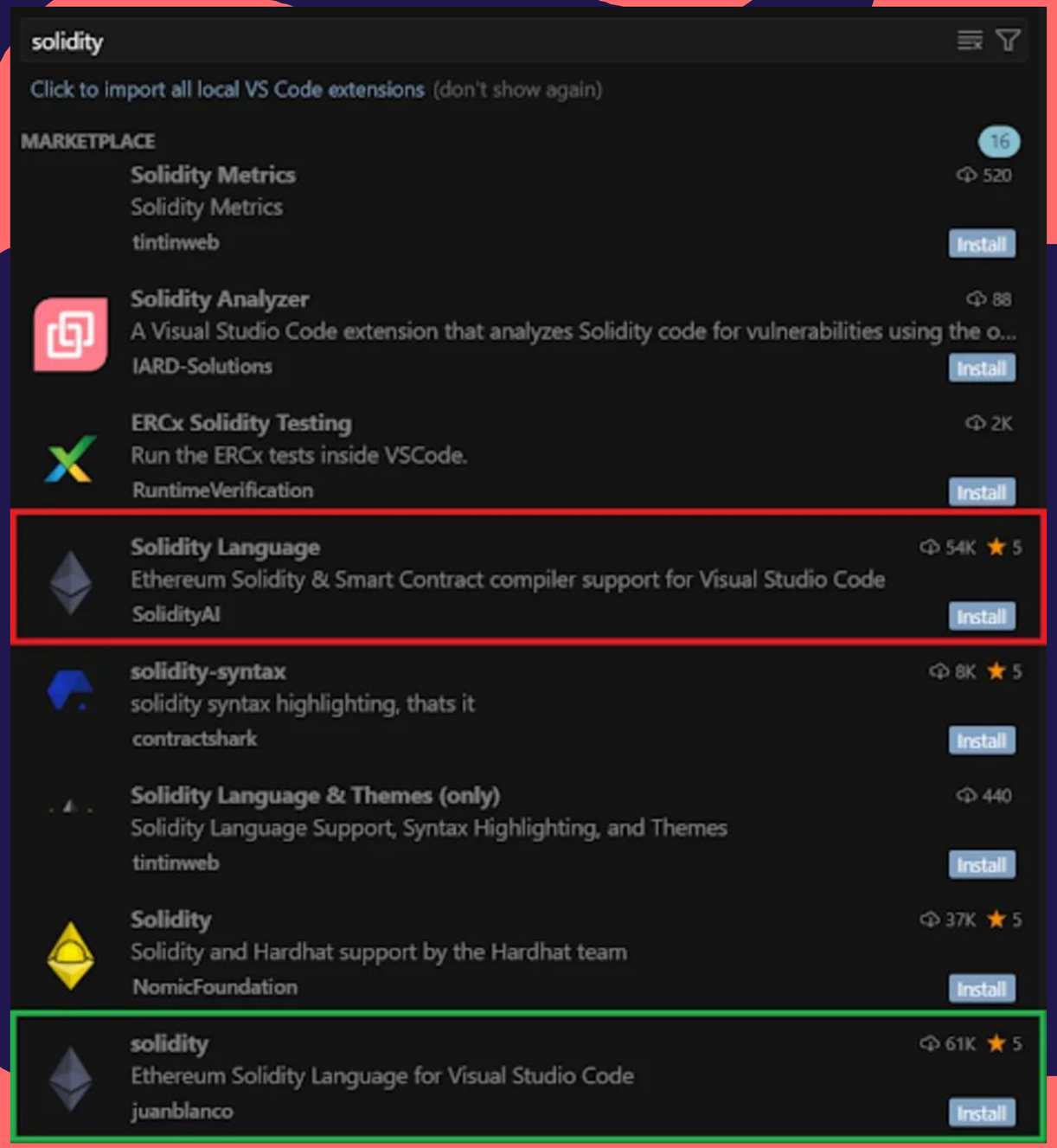


Image credit: https://opensourcemalware.com/blog/windows-infostealer-stubmaker-npm-ruby

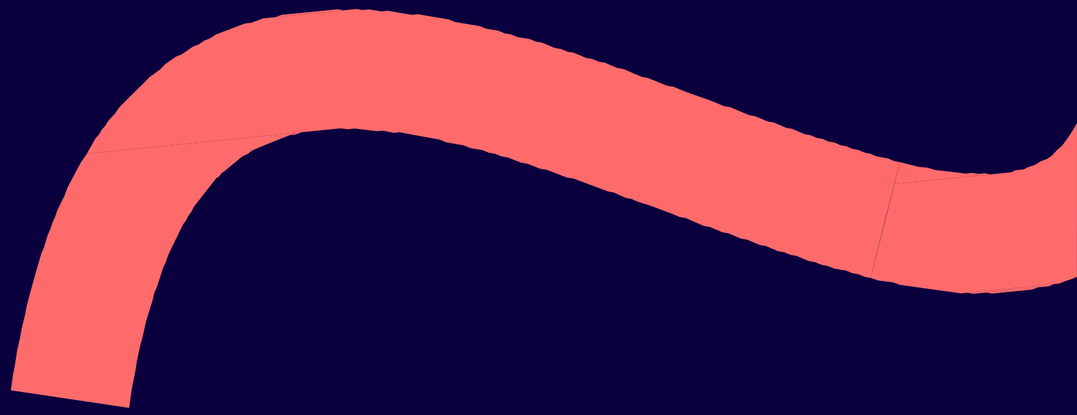
INFOBIP SHIFT

Infobip Shift 2026
Zadar, Croatia

Beware of evil extensions:



4) Recognizing AI generated code



Easy giveaways for AI generated code

- Emoji comments!
- Very linear comments (Step 1, Step 2…)
- Perfect formatting that would be difficult for a human
- Very perfect print statements with verbose language
- Redundant or unnecessary functions
- Lack of user comprehension about what the code does

Easy giveaways for AI generated code

- Crashes or fails without meaningful errors, no evidence of incremental development or debugging or unit tests
- Nonsensical imports
- Function stubs that don’t do anything
- No attempt to integrate with existing environment

INFOBIP SHIFT

Infobip Shift 2026
Zadar, Croatia

Examples:

```cpp
int main() {
    std::cout << "🎬 Starting Super Fun Data Analyzer v0.1! 🎬\n";

    auto records = loadRecords(); // load data 🗂️
    std::cout << "Loaded " << records.size() << " records! 🎉\n";

    auto analysis = analyzeRecords(records); // analyze data 🔍
    std::cout << "Analysis complete! ✅\n";

    saveResults(analysis); // save results 💾

    std::cout << "All done! 🎉 Have a nice day! 🌈\n";
    return 0;
}
```

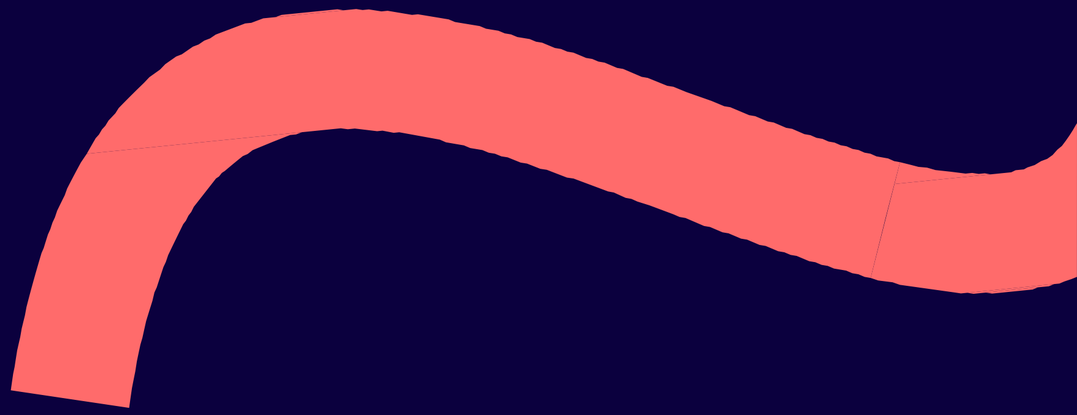
INFOBIP SHIFT

Infobip Shift 2026
Zadar, Croatia

Examples:

```
/*
  ___                _      ___         _
 | _ \___ __ _ _| |__ |   \ __ _| |_ __ _
 |  _/ -_) _` | / /   | |) / _` |  _/ _` |
 |_| \___\__,_|_\_\   |___/\__,_|\__\__,_|
*/

app.post('/process', (req, res) => {
    const { data } = req.body;

    // STEP 1: VALIDATION
    if (!Array.isArray(data)) return res.status(400).send("Invalid format");

    // STEP 2: TRANSFORMATION
    const clean = data.map(i => i.toString().trim());

    // STEP 3: FINALIZATION
    console.log(`Peak Data: Processed ${clean.length} items.`);
    res.send({ status: "success", count: clean.length });
});
```

5) Quiz time!



Spot the vulns!

```
var users = map[string]string{}

func register(username, password string) bool {
    if _, exists := users[username]; exists {
        return false
    }
    users[username] = password
    return true
}

func login(username, password string) bool {
    // TODO: implement login verification
    return false
}
```

INFOBIP SHIFT

Infobip Shift 2026
Zadar, Croatia

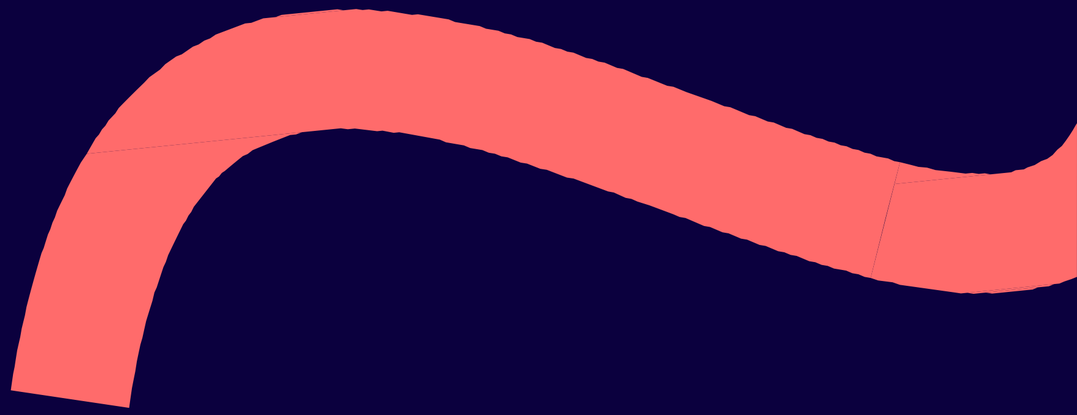
Spot the vulns!

```
app.post("/register", (req, res) => {
  const { username, password } = req.body;
  if (users.has(username)) return res.status(400).json({ error: "User exists" });
  users.set(username, hash(password));
  res.json({ message: "Registered" });
});

app.post("/login", (req, res) => {
  const { username, password, token } = req.body;

  if (!verifyLogin(username, password)) {
    return res.status(401).json({ error: "Invalid credentials" });
  }

  if (token !== "123456") {
    return res.status(401).json({ error: "Invalid MFA token" });
  }

  res.json({ message: "Authenticated" });
});
```

Spot the vulns!

```python
from fastapi import FastAPI, Query
import subprocess


app = FastAPI()


@app.get("/ping")
def ping(host: str = Query(...)):
    res = subprocess.run(f"ping -c 4 {host}", shell=True, capture_output=True, te
    return {"returncode": res.returncode, "output": res.stdout + res.stderr}
```

6) Resolving vulns



Resolve those vulns!

- Emphasize security from the first prompt
- Reference secure coding standards: OWASP cheatsheets
- Knowledge bases, instruction files, steering docs
- Code review with same and different models (Adversarial AI)
- Add MCP servers for security tools
- Add skills/plugins for security (but still verify!)
- Use ‘trust’ mode only in controlled envs

Secure from the first prompt:

I need a search function using nodejs. Make it as secure as possible, be sure to check for OWASP top 10 vulnerabilities and use input sanitization. Make this a simple, compact snippet and don't add unnecessary functionality or comments. Do a quality check to make...

To create a secure search function in Node.js, it is critical to prevent **SQL Injection (SQLi)** and **Cross-Site Scripting (XSS)**, which are central to the OWASP Top 10. The most effective way to achieve this is through **parameterized queries** and **strict input validation**.

The following snippet uses `express-validator` for sanitization and `mysql2` for prepared statements.

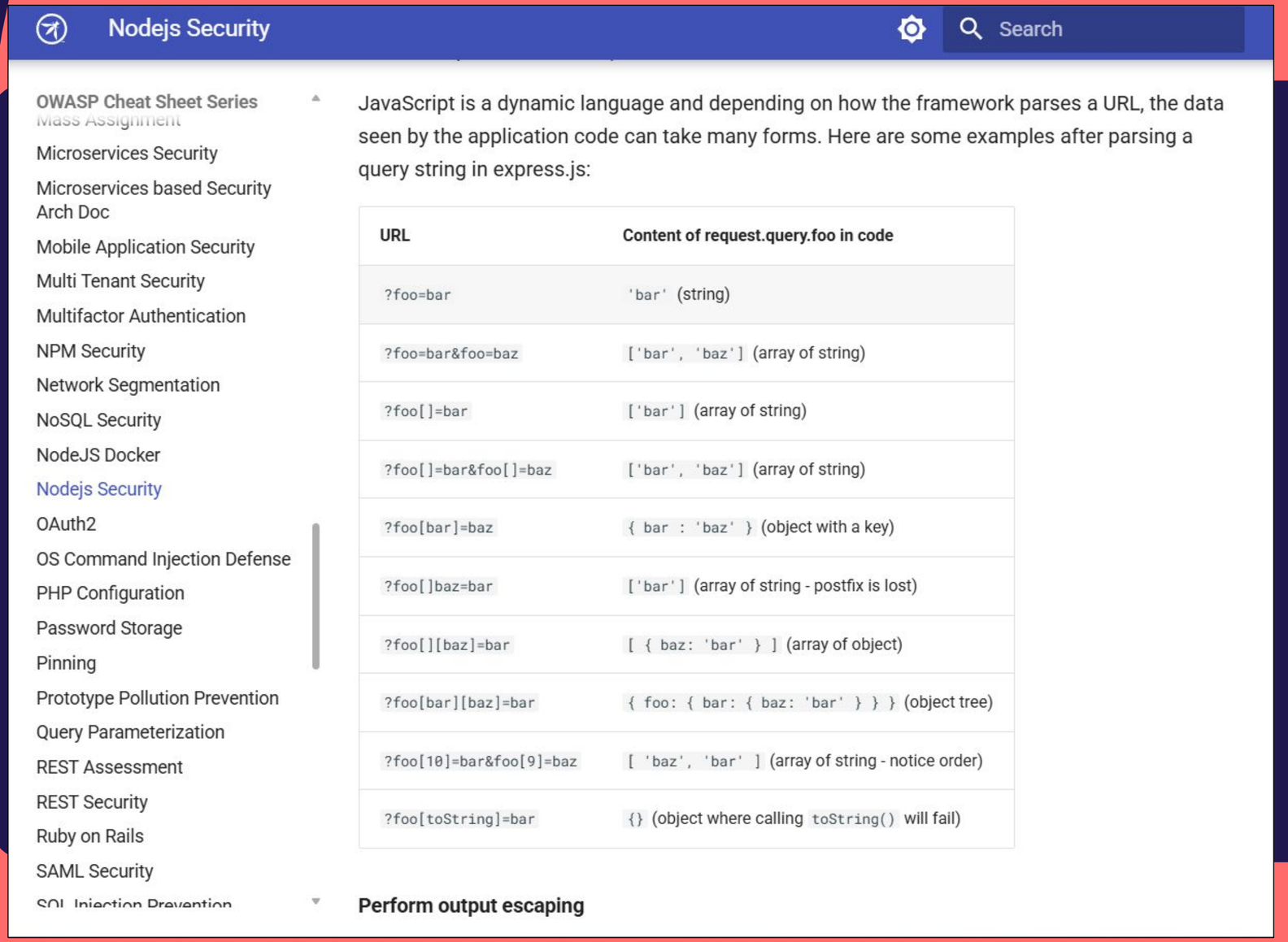
JavaScript

```
const express = require('express');
const { query, validationResult } = require('express-validator');
const mysql = require('mysql2/promise');
```

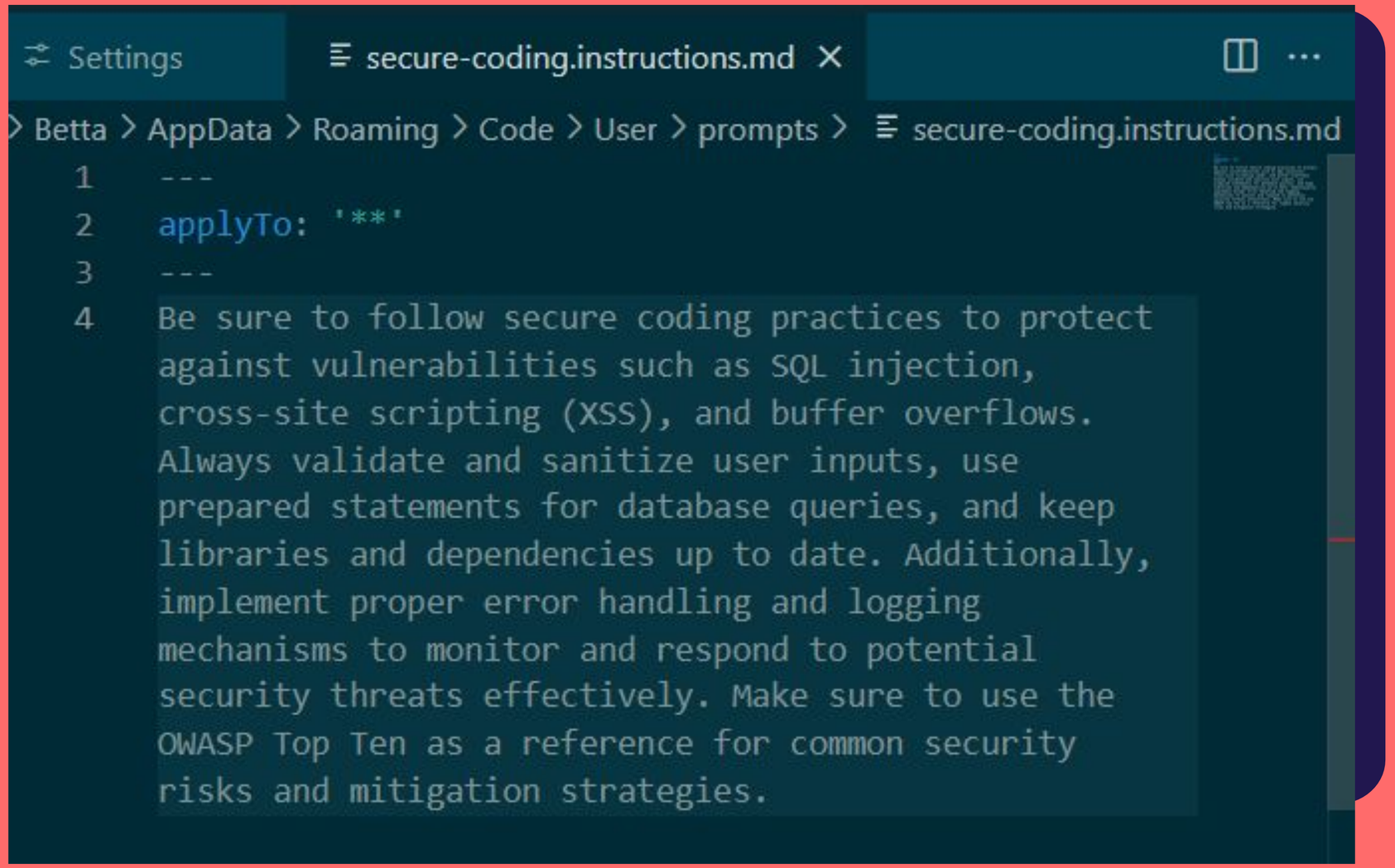
INFOBIP SHIFT

Infobip Shift 2026
Zadar, Croatia

OWASP Secure Coding Cheatsheets:



Add an instruction file or skills:

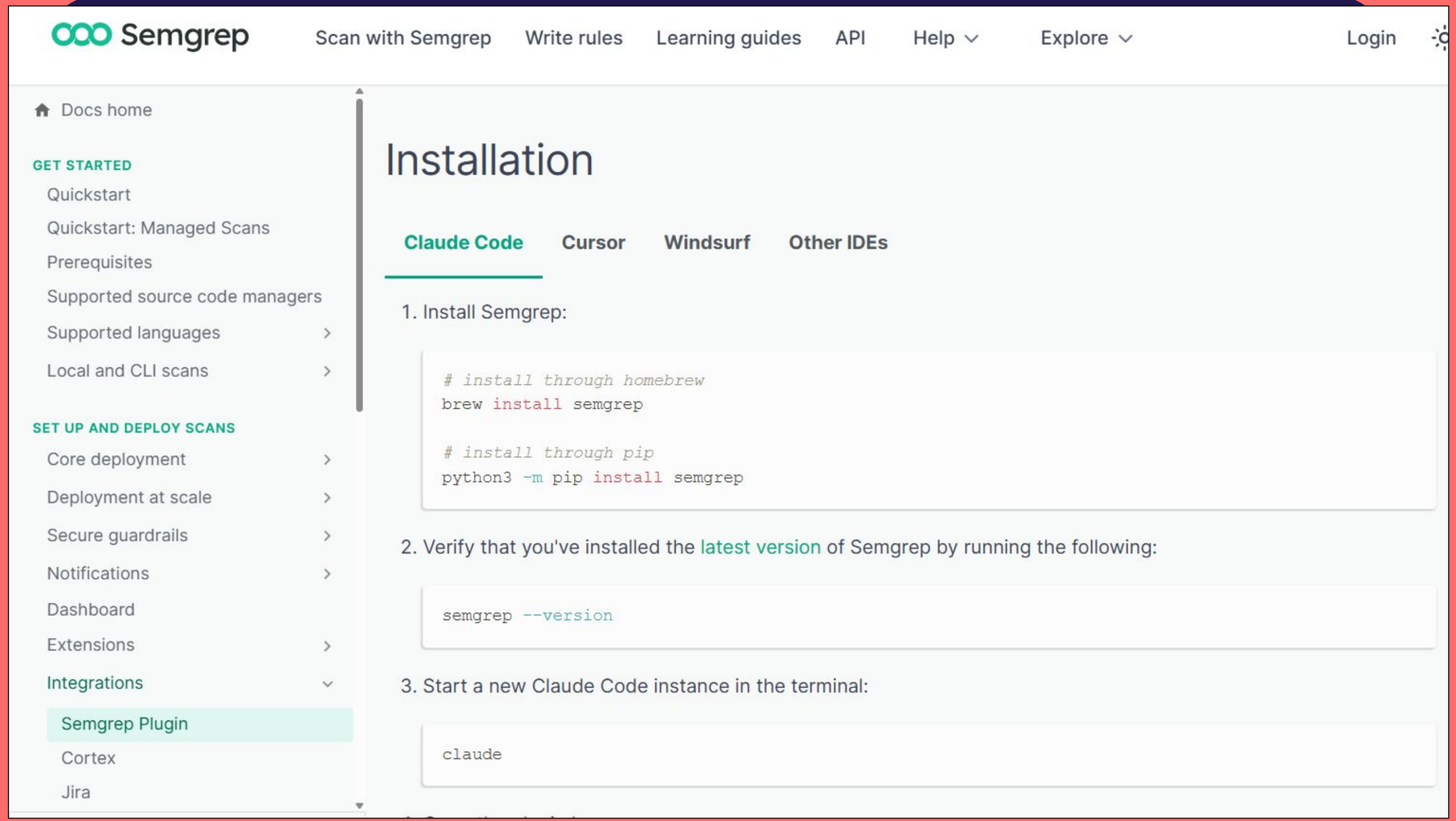


INFOBIP SHIFT

Infobip Shift 2026
Zadar, Croatia

Semgrep MCP:

https://semgrep.dev/docs/mcp

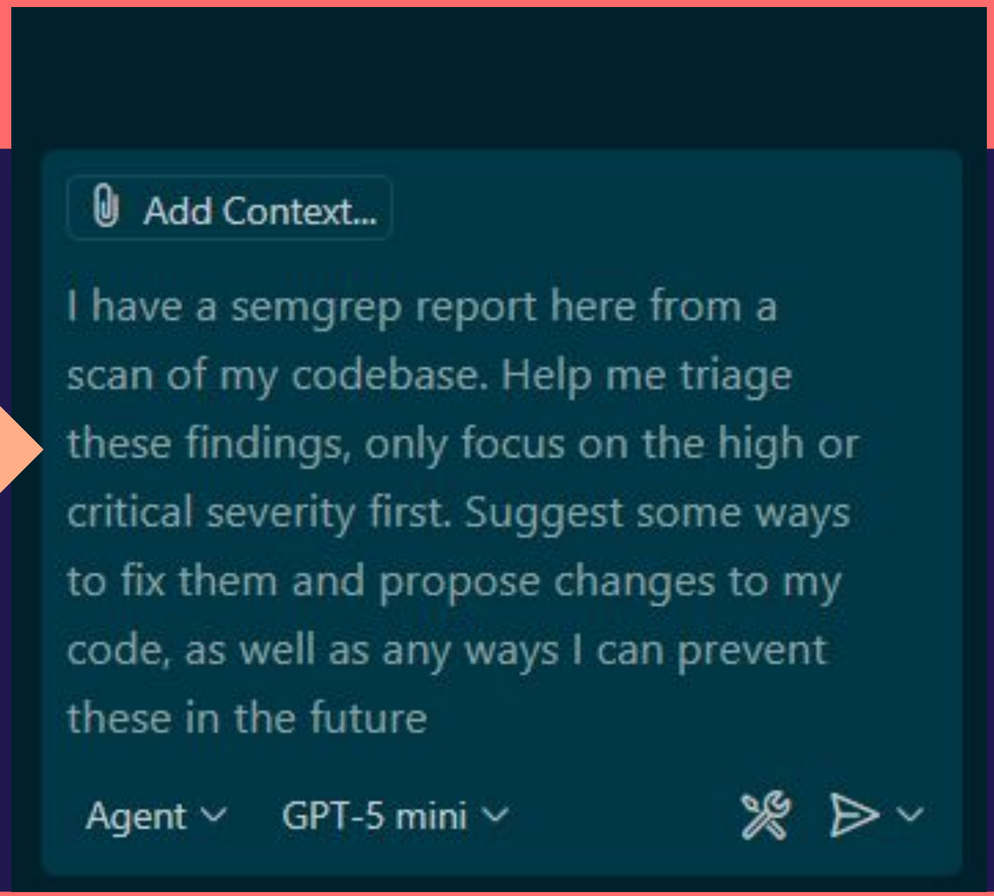


INFOBIP SHIFT

Infobip Shift 2026
Zadar, Croatia

Scan and vibe fixes:

```
~/apps/myapp: semgrep scan --config auto
Semgrep rule registry URL is https://semgrep.dev/registry.

Scanning across multiple languages:
    <multilang> |  54 rules × 36 files
             js | 179 rules ×  8 files
           json |   4 rules ×  3 files

──────────────────────────────────── 47/47 tasks 0:00:00


 Results

Findings:

  app.js
     javascript.express.security.audit.express-check-csurf-middleware-usage.express-check-csurf-
     middleware-usage
        A CSRF middleware was not detected in your express application. Ensure you are either using
        one  such as `csurf` or `csrf` (see rule references) and/or you are properly doing CSRF
        validation in your routes with a token or cookies.
        Details: https://sg.run/BxzR

         10¦ var app = express();


  bin/www
     problem-based-packs.insecure-transport.js-node.using-http-server.using-http-server
        Checks for any usage of http servers instead of https servers. Encourages the usage of https
        protocol instead of http, which does not have TLS and is therefore unencrypted. Using http
        can lead to man-in-the-middle attacks in which the attacker is able to read sensitive
```

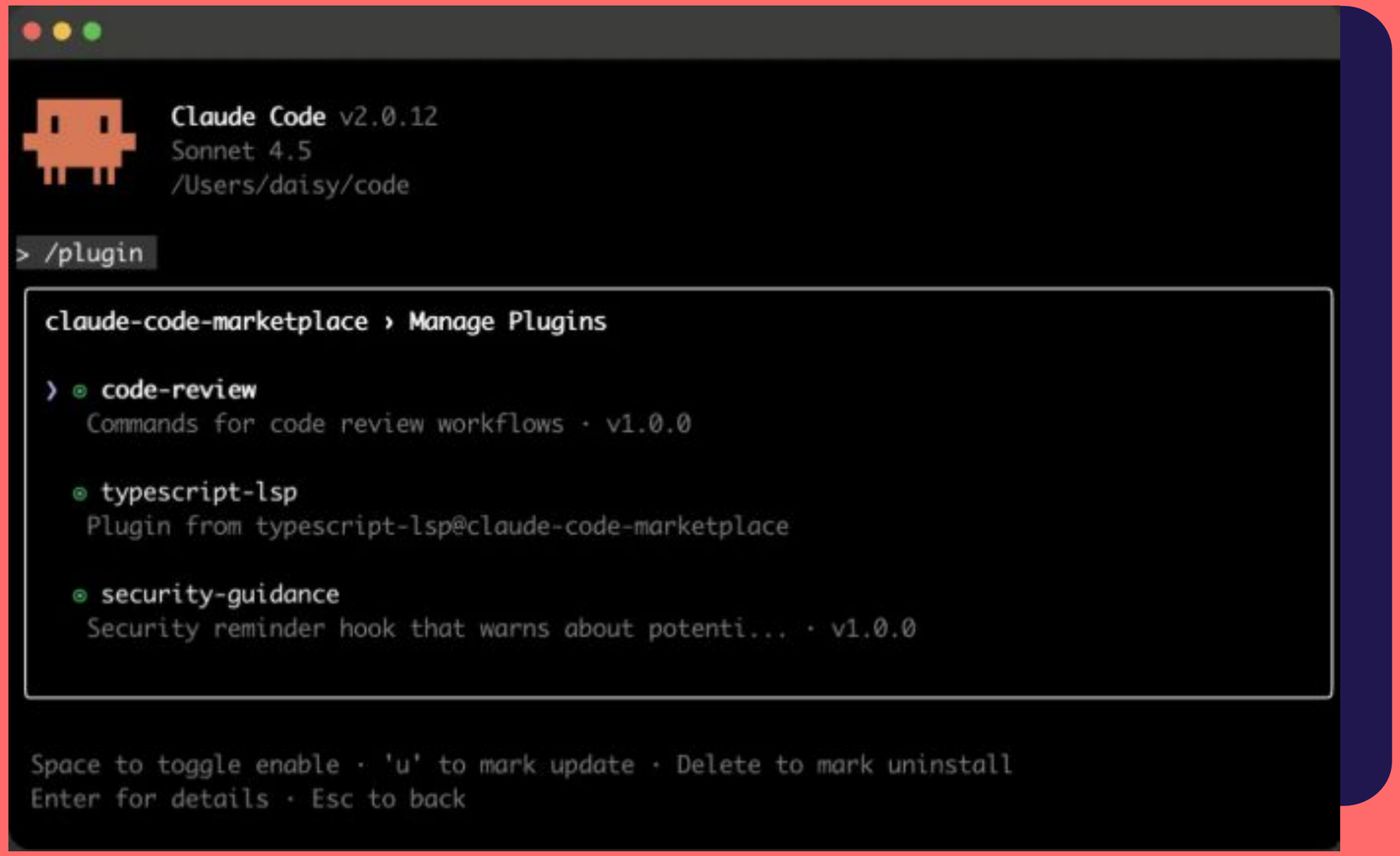


INFOBIP SHIFT

Infobip Shift 2026
Zadar, Croatia

Security plugins & skills:

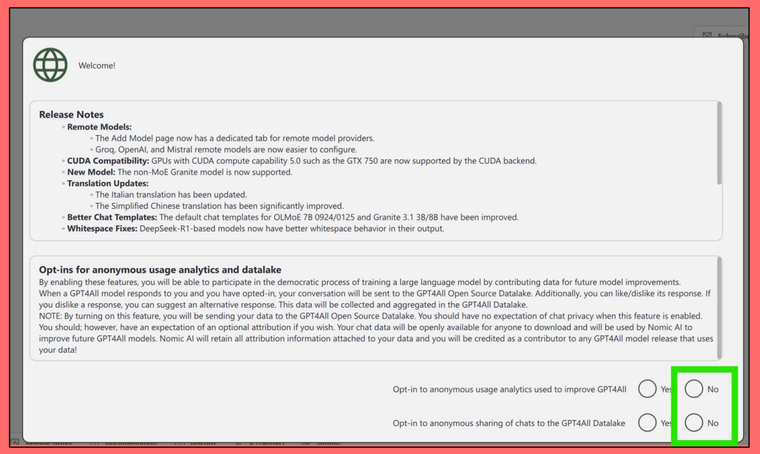
https://composio.dev/content/claude-code-plugin

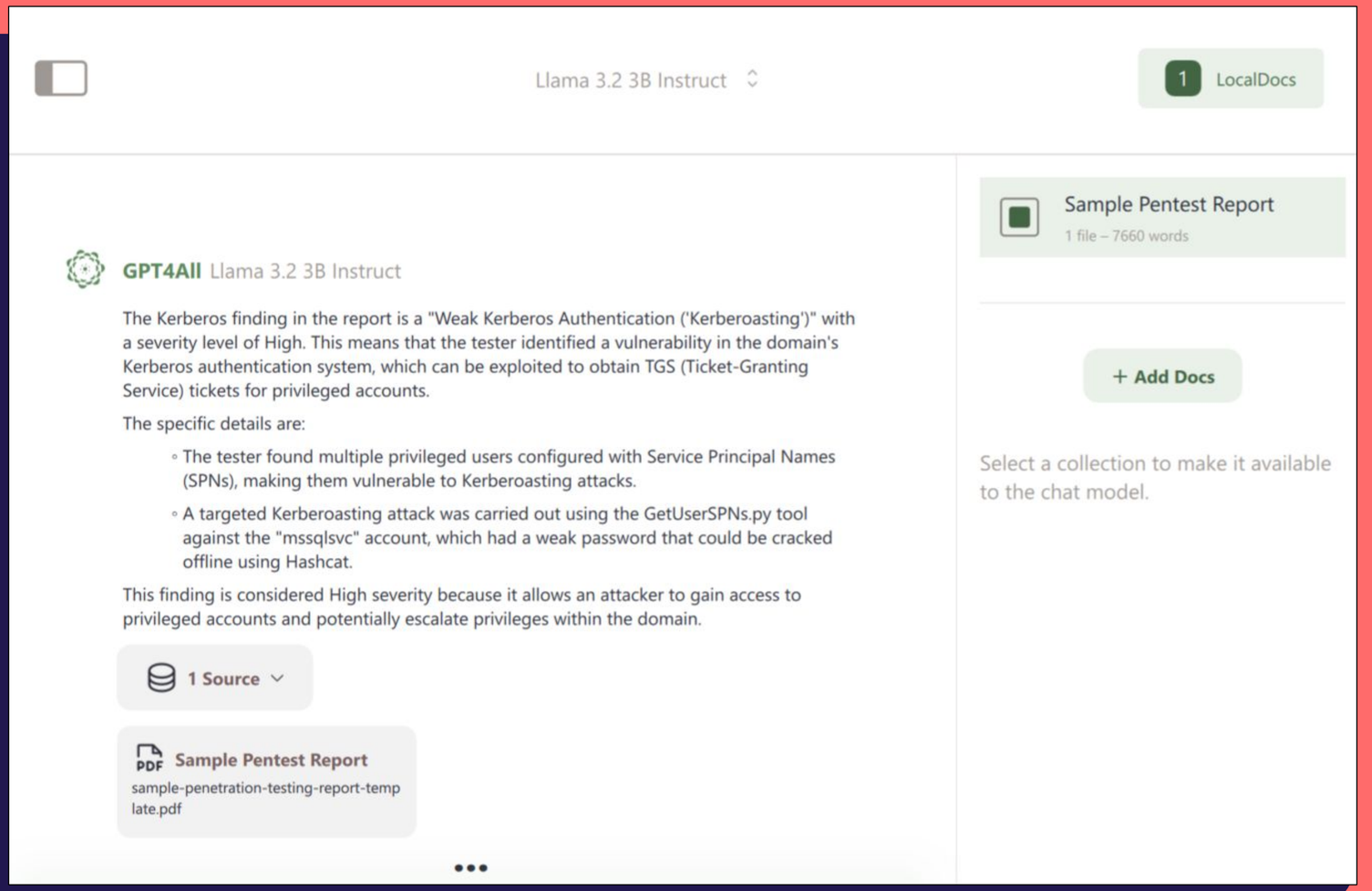


INFOBIP SHIFT

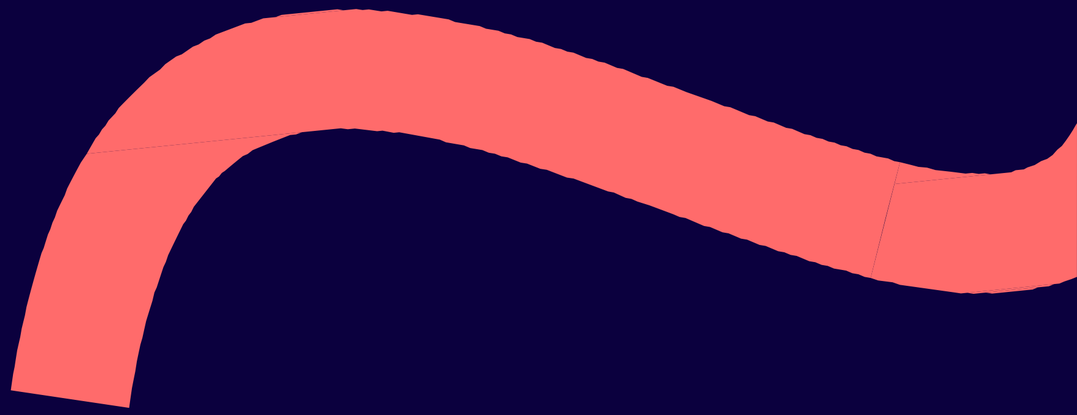
Infobip Shift 2026
Zadar, Croatia

Offline AI w/ Ollama or GPT4All:





7) AI-assisted coding



How to do ‘AI-assisted coding’

Infobip Shift 2026
Zadar, Croatia

- Instead of having the AI write everything, use it as a companion to help you troubleshoot tough errors
- Try to code out the key functions as you normally would manually, but then ask for help when you get tough errors or are exploring new tech stacks
- Get help with troubleshooting and ask for secure architecture advice
- Essentially: treat AI as an advisor instead of outsourcing all your work

Examples:

Help me troubleshoot this error. I am building an application with nodejs and I need to figure out where this issue came up:

All tests
passed./project/test/node_modules/mocha/lib/runner.js:873
throw err;
^Error [ERR_INTERNAL_ASSERTION]: This is caused by either a bug in Node.js or incorrect usage of Node.js internals.Please open an issue with this stack trace at https://github.com/nodejs/node/issues

at assert (internal/assert.js:14:11)
at process.target._send (internal/child_process.js:689:5)
at process.<anonymous>
(internal/child_process.js:646:12)

That particular error is a bit of a "blue screen of death" for Node.js processes. While it claims it could be a bug in Node itself, it is much more frequently caused by **attempting to send data to a child process that has already closed or is in an unstable state.**

Since this is happening during a Mocha test run, the culprit is usually how your tests handle asynchronous teardowns or IPC (Inter-Process Communication).

Examples:

Help me make this code more secure. I want to use an up to date cryptographic hashing method for storing the passwords and a better way of storing the passwords, with integrity checks. Also only give generic errors to the user, and have a lockout for too many attempts

```
def hash_password(password: str) -> str:
    return hashlib.md5(password.encode("utf-
8")).hexdigest()

@app.post("/register")
def register(user: User):
    if user.username in users:
        raise HTTPException(status_code=400,
detail="Username already exists")
```

Examples:

Give me a bash one liner that does a regex for internal IP addresses in a log file

You can use `grep -E` with a regex for private/internal IPs. Here's a compact **one-liner**:

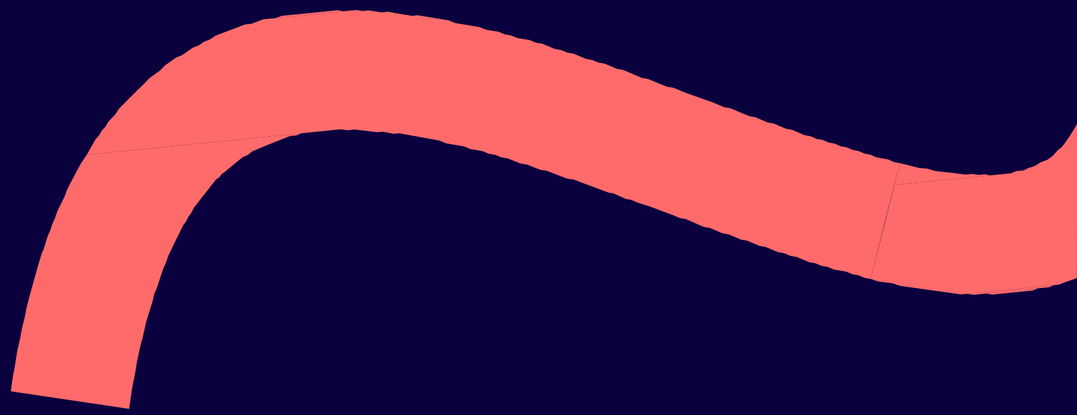
bash Copy code

```
grep -Eo '((10|127)\.([0-9]{1,3}\.){2}[0-9]{1,3}|172\.(1[6-9]|2[0-9]|3[0-1])\.([0-9]{1,3}\.)[0-9]
```

✅ **Explanation:**

- `-E` → extended regex
- `-o` → print only the matching part
- Matches:
  - `10.x.x.x`
  - `127.x.x.x` (loopback)
  - `172.16.x.x` – `172.31.x.x`
  - `192.168.x.x`

8) Further reading + workshop info



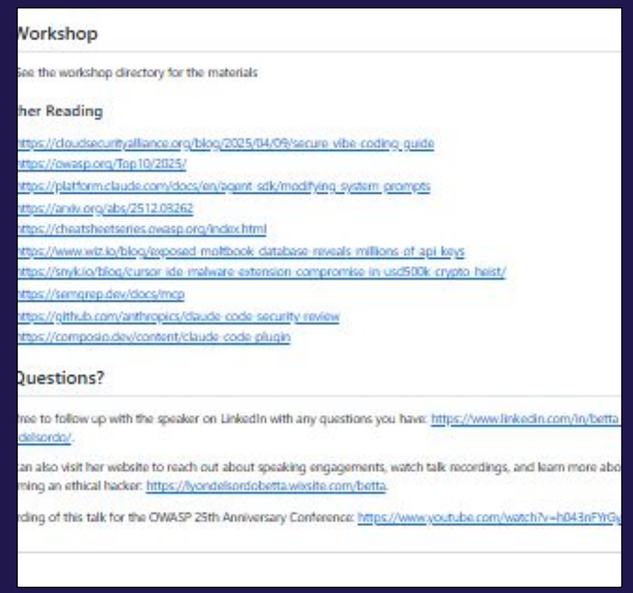
INFOBIP SHIFT

Infobip Shift 2026
Zadar, Croatia

Check out my workshop repo and let me know how it goes…

https://github.com/Betta-Lyon-Delsordo/insecure-vibes/

Scan for the workshop or go here



Articles & papers for further reading linked on the repo...

INFOBIP SHIFT

Infobip Shift 2026
Zadar, Croatia

Let’s connect!

Betta Lyon Delsordo

https://www.linkedin.com/in/betta-lyon-delsordo/